Software Design Document

for

<487-Group-Project>

Team: Team 9

Project: 487-Group-Project

Team Members:
Ivan Quintero
Alan Qiu
Benjamin Swanzey
Muath Alsawaier

Table of Contents

Table of Contents 2

Document Revision History 3

List of Figures 4

List of Tables 5

1. Introduction 6

1.1 Description 7

1.2 Architectural Design Goals 7

2. Software Architecture 7

2.1 Overview 8

2.2 Subsystem Decomposition 11

3. Subsystem Services 16

Document Revision History

Revision Number	Revision Date	Description	Rationale
1.0	3/29	Finalized documentation for submission	

This table provides a continuous record of document versions and must contain an entry for every version of this document that is published for review outside of your team. Revision numbering instructions:

- **The version of the document submitted for your gate review will be numbered 1.0.**
- **All versions that you produce for review before that time, such as that submitted to your peer review team, your sponsor, your first review draft, etc. will be numbered 0.x.**
- **The version submitted as your Baseline document (after your gate review and any required re-reviews) will be numbered 2.0.**
- **Any revisions made to the baseline will be numbered 2.x.**

List of Figures

Figure Number	Title	Page
Figure 2-1	MVC Architectural Pattern Component Diagram	7
Figure 2-2	Sequence Diagram: AI Spawns Enemies Use Case	8
Figure 2-3	Sequence Diagram: User Controls Player Movement Use Case	9
Figure 2-4	Sequence Diagram: Player Hit by Enemy Bullets Use Case	10
Figure 2-5	Object Pool Design Pattern: BulletManager Implementation	12
Figure 2-6	State Design Pattern: IScreen Implementation	13
Figure 2-7	Strategy Design Pattern: IShootingStrategy Implementation	14

All figures/diagrams/images included in your document must be numbered and titled. This list catalogues all such items.

Figure numbering instructions:

- **Each figure number will identify the section in which the figure appears and the sequential number of the figure in that section, separated by a hyphen. For example, the first figure in Section 1 Product Concept will be labeled "Figure 1-1", and the second figure in that section, if applicable, will be labeled "Figure 1-2". Likewise, the first figure in Section 3 Customer Requirements would be labeled "Figure 3-1".**
- **The title given each figure should be concise and descriptive.**

List of Tables

Table Number	Title	Page
Table 3-1	Subsystem Services and Interface Contracts	15

All ancillary tables/lists included in your document, with the exception of the Table of Contents, Revision History, List of Figures, and this table must be numbered and titled. This list catalogues all such items.

Table numbering instructions:

- **Each table number will identify the section in which it appears and the sequential number of the table in that section, separated by a hyphen. For example, the first Table in Section 1 Product Concept will be labeled "Table 1-1", and the second figure in that section, if applicable, will be labeled "Table 1-2". Likewise, the first figure in Section 3 Customer Requirements would be labeled "Table 3-1".**
- **The title given each table should be concise and descriptive.**

1. Introduction

1.1 Description

This project is a 2D “Bullet Hell” game built using the MonoGame framework. Inspired by the Touhou Project Series, the game challenges players to navigate around hundreds of projectiles and killing enemies ranging from low level grunts, medium level grunts, and bosses.

1.2 Architectural Design Goals

1.2.1 Performance (Latency & Resource Management)

- **Quality Attribute Scenario:** Under high-stress conditions (500+ active projectiles), the system must maintain a stable frame rate (60 FPS) to ensure input latency remains below 16ms.
- **Tactic: [Manage Resources - Bound Resources]**
    - **Description:** Following the performance tactics in Chapter 8, the system implements a **Resource Pool** via the `BulletManager`. By pre-allocating a fixed set of 500 bullets, we "bound" the memory resources used by the projectile subsystem.
    - **Rationale:** This avoids the overhead of frequent dynamic memory allocation and prevents the "Stop-the-World" latency spikes associated with .NET Garbage Collection, which would otherwise lag the game.

1.2.2 Availability (Fault Detection & Masking)

- **Quality Attribute Scenario:** If an invalid game state or null reference occurs during a screen transition, the system should catch the exception and default to a safe state (the Main Menu) rather than crashing.
- **Tactic: [Fault Detection - Exception Detection]**
    - **Description:** Using tactics from Chapter 5, the `Game1` controller and `ScreenManager` utilize structured exception handling and state validation during the `TransitionTo` method.
    - **Rationale:** In a game environment, "masking" a fault by returning the user to a stable menu is preferable to a hard crash, as it preserves the overall availability of the application and protects the integrity of the user's high score data.

2. Software Architecture

2.1 Overview

The system is designed using the **Model View Controller (MVC)** architectural pattern. This pattern was chosen to decouple the high-performance gameplay logic from the UI and rendering systems.

- **Rationale:** In a bullet hell game, the "Model" (bullet trajectories, collision logic) is computationally expensive. By separating it from the "View" (rendering code in `SimpleDrawer`), we can optimize the game loop and modify the visual assets without risking the integrity of the movement and physics logic.

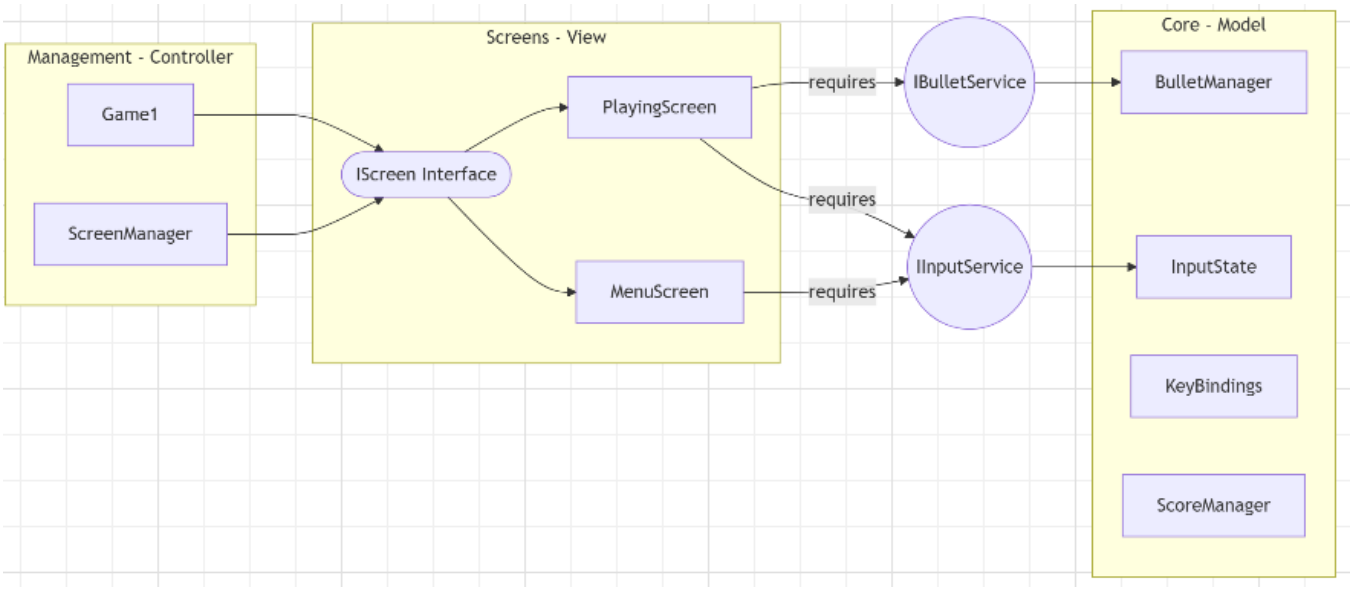

Figure 2-1: MVC Architectural Pattern Component Diagram

2.1.1: Use Case 1 - AI spawns enemies

- **Actor:** `EnemyFactory` (or `Timer`)
- **Logic:** The `Game1` update loop checks the spawn timer. When triggered, it calls the `EnemyFactory` to create an enemy instance with a specific `IShootingStrategy` and adds it to the active entities list.

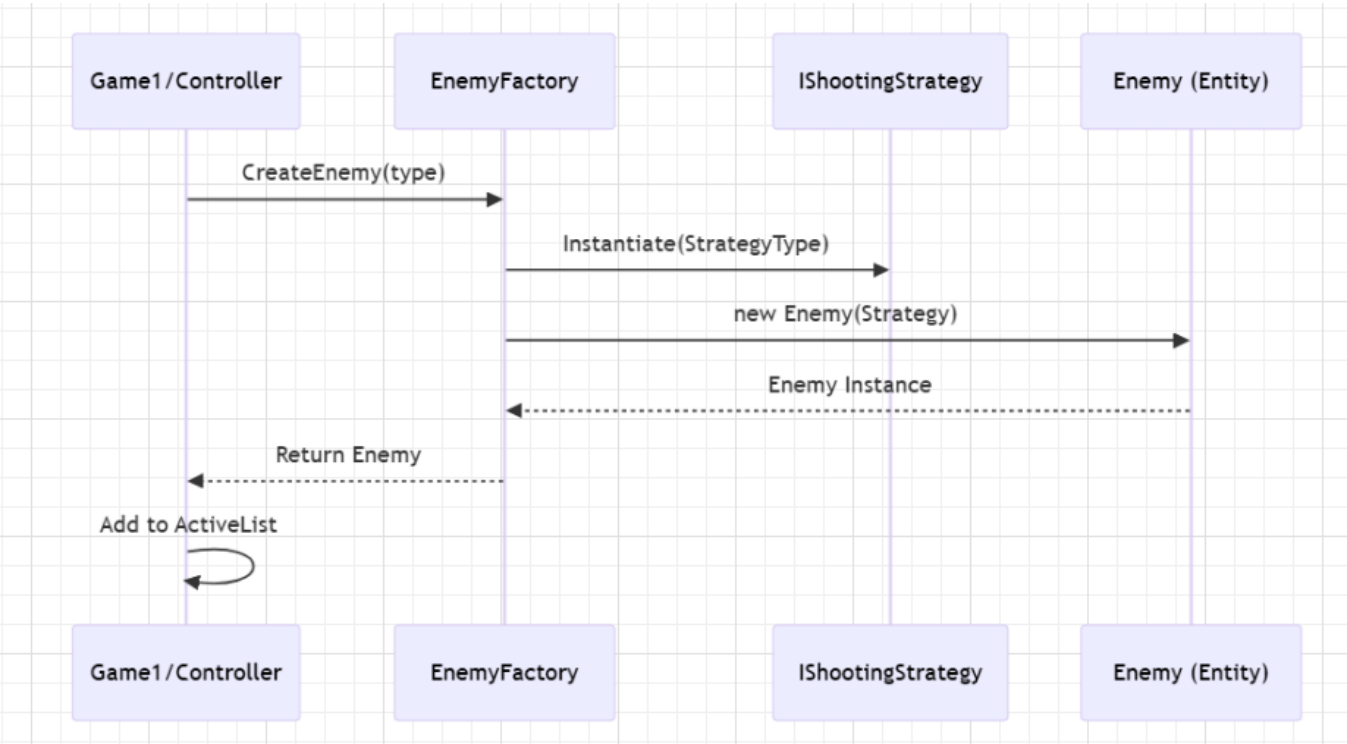

Figure 2-2: Sequence Diagram: AI Spawns Enemies Use Case

2.1.2: Use Case 2 - User controls player to move

- **Actor:** `Player`
- **Logic:** `Game1` calls `Update` on the `PlayingScreen`. The screen polls the `InputState` (Core). Based on the returned `KeyBindings`, the `Player` object updates its `Position` vector.

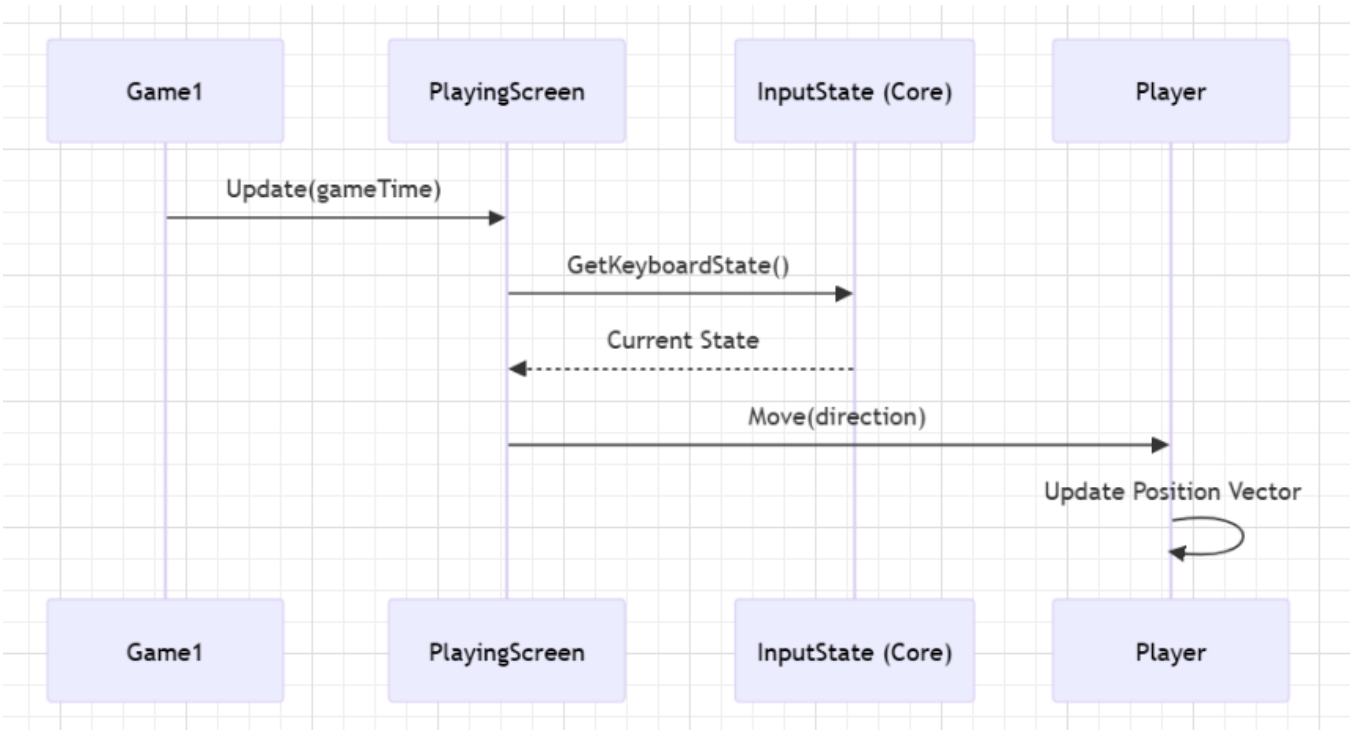

Figure 2-3: Sequence Diagram: User Controls Player Movement Use Case

2.1.3: Use Case 3 - Player is hit by enemy bullets

- **Actor:** `BulletManager`
- **Logic:** The `BulletManager` iterates through active bullets. It checks the `BoundingBox` of a bullet against the `Player` hitbox. If they intersect, the `Player` calls `TakeDamage()`, and the bullet is returned to the pool (`IsAlive = false`).

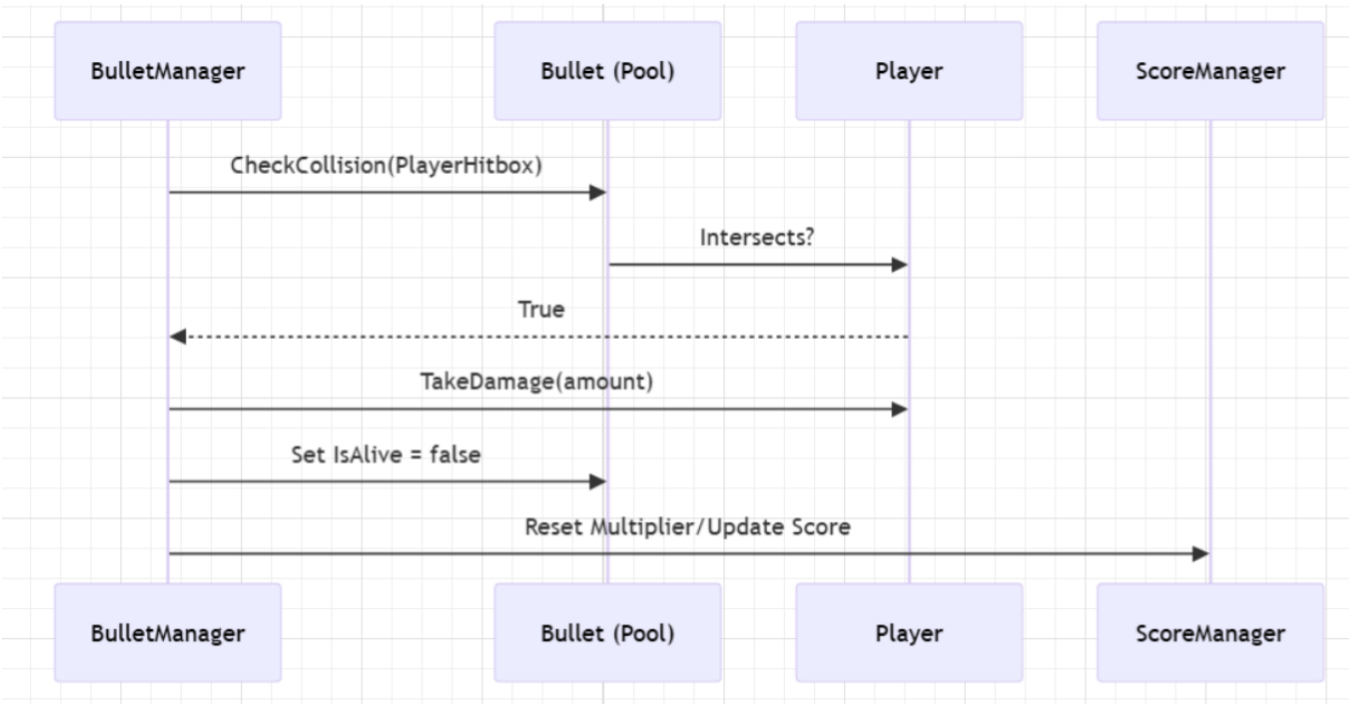

Figure 2-4: Sequence Diagram: Player Hit by Enemy Bullets Use Case

2.2 Subsystem Decomposition

2.2.1 Management Subsystem (Controller)

The Management subsystem acts as the central orchestrator for the application's lifecycle. It is responsible for initializing the MonoGame framework, polling hardware via the Core subsystem, and managing transitions between different game states.

- **Primary Classes:** `Game1`, `ScreenManager`.
- **Responsibilities:**
  - Implementing the main `Update` and `Draw` loops.
  - Managing the `IScreen` stack to handle menus, gameplay, and pausing.
  - Coordinating global configuration settings via `GameConfig`.

2.2.2 Screens Subsystem (View)

The Screens subsystem encapsulates visual and behavioral logic for specific game states. Each screen is a self-contained unit that defines how entities are rendered and how the UI reacts to user input.

- **Primary Classes:** `IScreen` (Interface), `PlayingScreen`, `MenuScreen`, `KeyConfigScreen`, `SimpleDrawer`.
- **Responsibilities:**
  - Defining distinct rendering logic for gameplay vs. UI-heavy menus.
  - Utilizing the `SimpleDrawer` utility to render primitive shapes (hitboxes, health bars).
  - Mapping logical game actions for visual feedback.

2.2.3 Core Subsystem (Model)

The Core subsystem provides the foundational services required for a high-performance bullet hell game. It manages the "truth" of the game world, including the high-speed physics of projectiles and the state of the player's input and score.

- **Primary Classes:** `BulletManager`, `InputState`, `KeyBindings`, `ScoreManager`.
- **Responsibilities:**
  - **Resource Management:** Maintaining an Object Pool of 500 bullets to ensure stable performance.
  - **Input Abstraction:** Mapping physical hardware keys to logical game actions.
  - **State Tracking:** Real-time calculation of score and projectile positions.

2.2.4 Design Patterns

2.2.4.1 Object Pool Pattern (BulletManager)

Implementation: The `BulletManager` follows the Object Pool pattern by pre-allocating a static list of 500 `Bullet` objects at initialization. Instead of creating new objects, the system calls `GetInactiveBullet()` to repurpose a previously "dead" bullet.

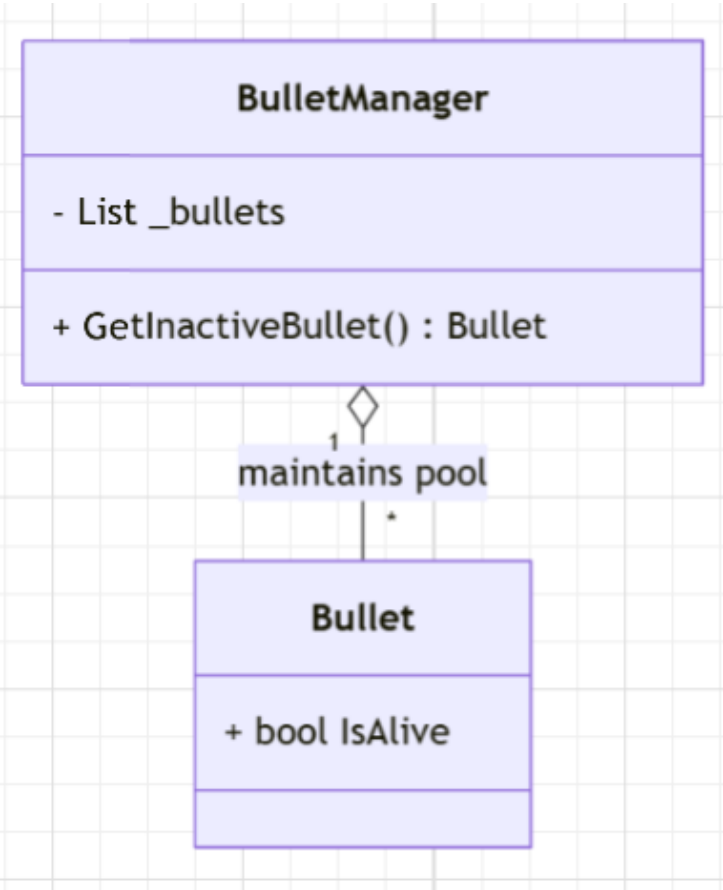

Figure 2-5: Object Pool Design Pattern: BulletManager Implementation

Class Diagram Logic:

- **Manager:** `BulletManager` contains a private `List<Bullet>`.
- **Product:** `Bullet` contains an `IsAlive` flag.
- **Operations:** `BulletManager.Update()` iterates through the list; `FireBullet()` activates a member of the pool.

Deviation: In a classic Object Pool, the pool might grow dynamically if exhausted. Our implementation uses a Fixed-Size Pool of 500. If the pool is full, new fire requests are ignored.

2.2.4.2 State Pattern (IScreen)

Implementation: By interacting with the `IScreen` interface rather than concrete classes, the main game loop remains unchanged regardless of whether the user is in the `MenuScreen` or the `PlayingScreen`.

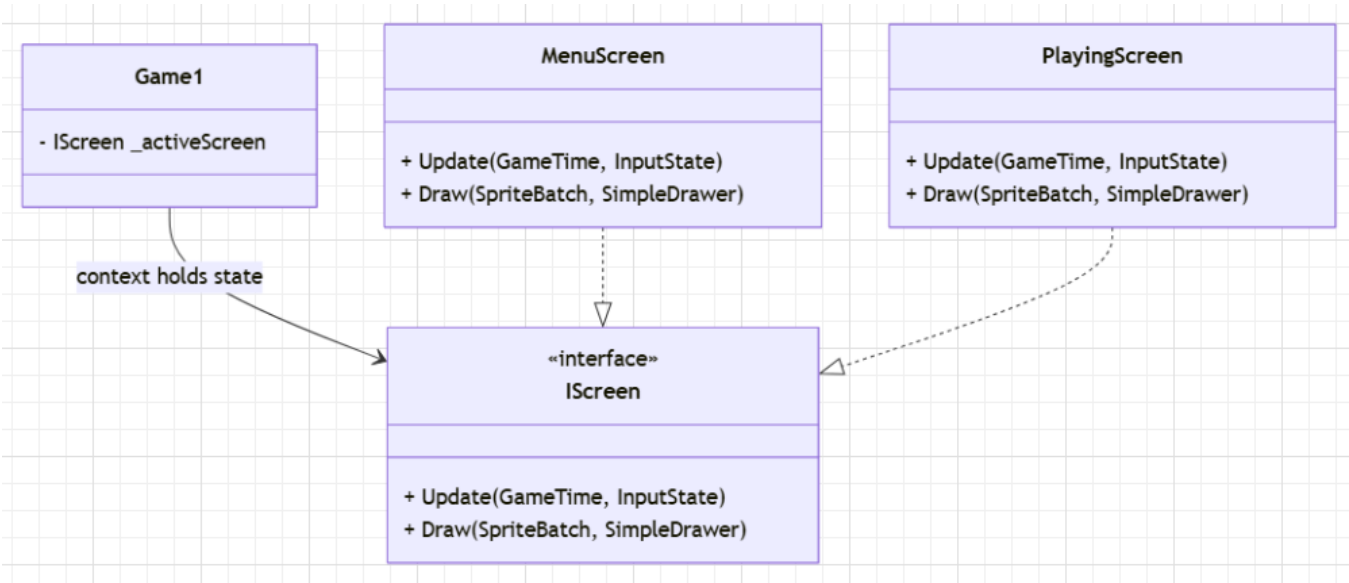

Figure 2-6: State Design Pattern: IScreen Implementation

Class Diagram Logic:

- **Context:** `Game1` holds a reference to an `IScreen _activeScreen`.
- **State Interface:** `IScreen` defines `Update(GameTime)` and `Draw(SpriteBatch)`.
- **Concrete States:** `MenuScreen` and `PlayingScreen` implement the interface.

2.2.4.3 Strategy Pattern (IShootingStrategy)

Implementation: To allow enemies to have diverse firing behaviors without code duplication, we use the Strategy pattern. Each `Enemy` object is composed of an `IShootingStrategy`. This allows an enemy to switch from a "Targeted" shot to a "Random Scatter" shot simply by swapping the strategy class.

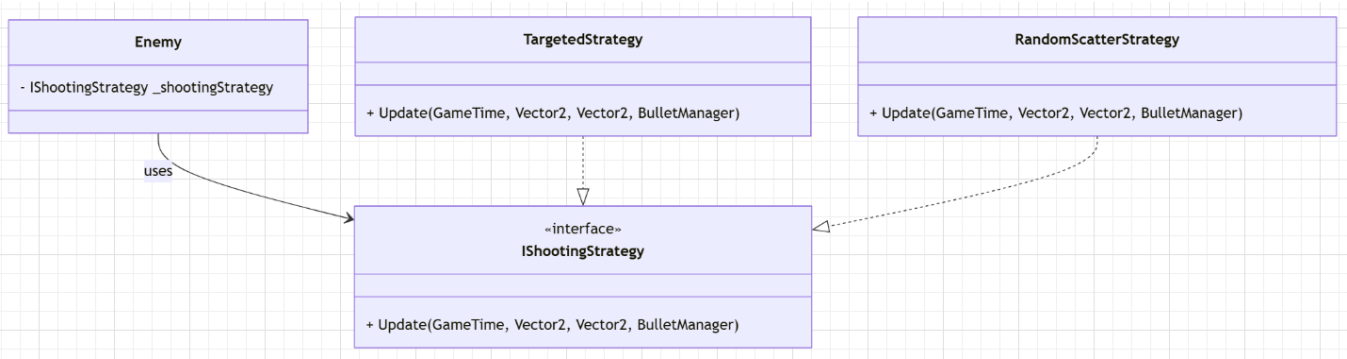

Figure 2-7: Strategy Design Pattern: IShootingStrategy Implementation

Class Diagram Logic:

- **Context:** `Enemy` class.
- **Strategy Interface:** `IShootingStrategy`.
- **Concrete Strategies:** `TargetedStrategy`, `RandomScatterStrategy`, `LaserBeamStrategy`.

3. Subsystem Services

Subsystem	Services (Interfaces)	Provided by (Class)	Required by (Subsystem/Class)	Description
Screens	IScreen	PlayingScreen, MenuScreen	Management / Game1	Defines the standard Update and Draw contract that the main game loop uses to render the current game state.
Core	IInputService	InputState	Screens / PlayingScreen, MenuScreen	Provides the current state of keyboard/controller input to allow entities to move or menu options to be selected.
Core	IBulletService	BulletManager	Screens / PlayingScreen	Manages the 500-bullet object pool; allows the gameplay view to request projectile spawning and collision updates.
Core	IScoreService	ScoreManager	Screens / GameOverScreen	Tracks player points and high scores to be displayed on the UI after a game session ends.

Table 3-1: Subsystem Services and Interface Contracts